

■ Guide Ultime : Configuration d'Expo Updates (OTA) sur Android de A à Z

Objectif : Maîtriser le déploiement de mises à jour Over-The-Air (OTA) sans fil sur Android pour les projets React Native avec Expo (Bare Workflow ou Prebuild) sans subir les erreurs courantes de configuration native.

■ Table des Matières

- [Concepts Clés & Différence Binary vs JS Bundle](#)
- [Étape 1 : Configuration d'app.json & eas.json](#)
- [Étape 2 : Préparation du Projet Natif Android \(Prebuild & Manifest\)](#)
- [Étape 3 : Distinguer les Builds Debug vs Release](#)
- [Étape 4 : Déploiement & Publication \(**eas update**\)](#)
- [Étape 5 : Gestion du Téléchargement en Code JS \(**Updates.checkForUpdateAsync**\)](#)
- [Checklist Anti-Erreurs & Troubleshooting](#)

1. Concepts Clés & Différence Binary vs JS Bundle

Pour comprendre le fonctionnement d'Expo Updates (OTA), il faut séparer deux éléments majeurs d'une application React Native :

■ APPLICATION EXPO ANDROID ■

■ 1. LE CODE NATIF (C++) ■ 2. LE BUNDLE JAVASCRIPT ■ ■ (Java, Kotlin, Gradle, C++) ■ (Code React, TS, Assets) ■

■ • Nécessite une recompilation APK ■ • Peut être mis à jour sans fil ■ ■ • Changement dans android/ ■ • Modifié via `eas update` ■ ■ • Ajout de modules C++/Nitro ■ • Mis à jour en 30 secondes ■

Le rôle de `runtimeVersion`

Le champ `runtimeVersion` définit un contrat entre le code natif de l'APK et le bundle JS.

Si le **runtimeVersion** de votre update publié sur EAS ne correspond pas au **runtimeVersion** configuré dans l'APK natif installé sur le téléphone, le moteur d'updates **refusera le téléchargement** afin d'éviter un crash natif.

2. Étape 1 : Configuration d'app.json & eas.json

A. Dans `app.json`

Assurez-vous que votre fichier `app.json` comporte la configuration suivante :

```
{ "expo": { "name": "CombiStore", "slug": "CombiStore", "version": "1.0.0", "runtimeVersion": {
  "policy": "appVersion" }, "plugins": [ "expo-router", [ "expo-updates", { "username":
  "votre-nom-utilisateur-expo" } ] ], "extra": { "eas": { "projectId": "VOTRE-PROJECT-ID-EAS" }
}, "updates": { "enabled": true, "checkAutomatically": "ON_LOAD", "fallbackToCacheTimeout":
5000, "url": "https://u.expo.dev/VOTRE-PROJECT-ID-EAS" } } }
```

B. Dans `eas.json`

Définissez les canaux d'updates pour chaque profil de build :

```
{ "cli": { "version": ">= 5.9.3" }, "build": { "preview": { "distribution": "internal",
"channel": "preview", "android": { "buildType": "apk" } }, "production": { "channel":
"production", "android": { "buildType": "app-bundle" } } }
```

3. Étape 2 : Préparation du Projet Natif Android (Prebuild & Manifest)

A. Exécuter le Prebuild Expo

Pour que les plugins Expo injectent les configurations automatiquement dans le dossier Android :

```
npx expo prebuild --platform android
```

B. Les Métadonnées Critiques dans `AndroidManifest.xml`

Dans `android/app/src/main/AndroidManifest.xml`, sous la balise `<application>` :

```
<application android:name=".MainApplication" ...> <!-- 1. Activer le moteur d'updates -->
<meta-data android:name="expo.modules.updates.ENABLED" android:value="true"/> <!-- 2. Définir
le comportement de vérification au lancement --> <meta-data
android:name="expo.modules.updates.EXPO_UPDATES_CHECK_ON_LAUNCH" android:value="ALWAYS"/> <!--
3. Temps d'attente maximum au boot pour télécharger l'update (en ms) --> <meta-data
android:name="expo.modules.updates.EXPO_UPDATES_LAUNCH_WAIT_MS" android:value="5000"/> <!-- 4.
URL de votre projet EAS --> <meta-data android:name="expo.modules.updates.EXPO_UPDATE_URL"
android:value="https://u.expo.dev/VOTRE-PROJECT-ID-EAS"/> <!-- 5. Référence de la version
runtime --> <meta-data android:name="expo.modules.updates.EXPO_RUNTIME_VERSION"
android:value="@string/expo_runtime_version"/> <!-- 6. PIÈGE CRITIQUE #1 : En-tête HTTP JSON
pour spécifier le canal --> <meta-data
android:name="expo.modules.updates.UPDATES_CONFIGURATION_REQUEST_HEADERS_KEY"
android:value="{&quot;expo-channel-name&quot;:&quot;preview&quot;}" /> </application>
```

■■ ATTENTION AU PIÈGE DU CANAL !

La balise `<meta-data android:name="expo.modules.updates.EXPO_CHANNEL_NAME" .../>` n'est **PAS** lue par le SDK Android natif d'Expo ! Le nom du canal (**preview**) **DOIT** être fourni dans l'en-tête JSON `UPDATES_CONFIGURATION_REQUEST_HEADERS_KEY`.

C. La ressource String dans `strings.xml`

Vérifiez que `android/app/src/main/res/values/strings.xml` contient la valeur de version runtime :

```
<resources> <string name="app_name">CombiStore</string> <string
name="expo_runtime_version">1.0.0</string> </resources>
```

4. Étape 3 : Distinguer les Builds Debug vs Release

Pourquoi `assembleDebug` ne reçoit JAMAIS les updates OTA ?

Par conception, un APK compilé en mode **Debug** (`./gradlew assembleDebug`) se connecte au serveur JS de développement local (Metro sur `localhost:8081`). Le moteur d'updates natif y est volontairement désactivé.

Comment compiler un APK Release prêt pour l'OTA en local ?

Créez un script PowerShell `build.ps1` à la racine de votre projet :

```
$env:JAVA_HOME="C:\Program Files\Android\Android Studio\jbr"
$env:ANDROID_HOME="C:\Users\AndySmart\AppData\Local\Android\Sdk" cd android ./gradlew.bat
assembleRelease > ../build_log.txt 2>&1 Write-Host "Build terminé ! APK disponible dans :
android/app/build/outputs/apk/release/app-release.apk"
```

L'APK généré sous `android/app/build/outputs/apk/release/app-release.apk` intègre le moteur `expo-updates` actif et prêt à recevoir les mises à jour sans fil.

5. Étape 4 : Déploiement & Publication (`eas update`)

Pour publier une nouvelle version JavaScript sans réinstaller l'APK :

```
npx eas-cli update --branch preview --message "Mise à jour des composants et correctifs"
```

Ce qui se passe lors de la publication :

- Expo CLI prépare et compense les bundles JS (Hermes Bytecode `.hbc`).
- EAS télécharge les assets et le bundle vers les serveurs cloud Expo.
- EAS assigne l'update au canal spécifié (ex: `preview`) avec le `runtimeVersion 1.0.0`.

6. Étape 5 : Gestion du Téléchargement en Code JS (`Updates.checkForUpdateAsync`)

Pour gérer les alertes de mise à jour dans l'interface de l'application, créez un service `src/services/updatesService.ts` :

```
import * as Updates from 'expo-updates'; import { Alert } from 'react-native'; export async
function checkForAppUpdates(): Promise<void> { // Ne pas exécuter en mode développement Metro
local if (__DEV__) { return; } try { const update = await Updates.checkForUpdateAsync(); if
(update.isAvailable) { // Téléchargement du bundle JS en arrière-plan await
Updates.fetchUpdateAsync(); Alert.alert( 'Mise à jour disponible ■', 'Une nouvelle version est
prête. Voulez-vous la charger maintenant ?', [ { text: 'Redémarrer maintenant', onPress: async
() => { try { await Updates.reloadAsync(); } catch (e) { console.error('Erreur lors du
rechargement:', e); } }, { text: 'Plus tard', style: 'cancel', }, ] ); } } catch (error) {
console.log('Vérification OTA ignorée ou déjà à jour:', error); } }
```

Appelez ce service dans votre layout racine `app/_layout.tsx` :

```
useEffect(() => { if (fontsLoaded) { SplashScreen.hideAsync(); checkForAppUpdates(); } },
[fontsLoaded]);
```

■ **Remarque importante sur `LAUNCH_WAIT_MS`** : Si `EXPO_UPDATES_LAUNCH_WAIT_MS` est configuré à **5000** (5s), Android télécharge l'update **pendant le Splash Screen**. L'app démarrera directement sur la nouvelle version et `update.isAvailable` retournera **false** car l'app est **déjà à jour dès l'ouverture** !

7. ■■ Checklist Anti-Erreurs & Troubleshooting

Symptôme / Erreur	Cause Racine	Solution
L'APK ne reçoit aucune update	L'APK a été généré avec <code>assembleDebug</code> .	Utiliser <code>./gradlew assembleRelease</code> via <code>build.ps1</code> .
Serveur EAS répond 0 update available	Header de canal manquant dans <code>AndroidManifest.xml</code> .	Utiliser <code>UPDATES_CONFIGURATION_REQUEST_HEADERS_KEY</code> avec <code>{"expo-channel-name":"preview"}</code> .
Silent failure au boot	<code>EXPO_RUNTIME_VERSION</code> pointe vers un nom manquant dans <code>strings.xml</code> .	Vérifier que <code><string name="expo_runtime_version">1.0.0</string></code> existe.
Update rejetée sur le téléphone	Incompatibilité de <code>runtimeVersion</code> entre l'APK et l'update.	Vérifier que le <code>runtimeVersion</code> dans <code>app.json</code> match celui du build.
Pas d'alerte JS affichée	L'update a été pré-téléchargée pendant le Splash Screen.	L'application est déjà à jour ! <code>update.isAvailable</code> est donc false .

Guide rédigé pour les développeurs React Native / Expo sur Android.